



400-650-7088

***Tong*RDS** V2.2

开发手册

版本：2218A01

2025 年 9 月

声明

版权

Copyright ©2025 北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（2218A01）适用于 TongRDS V2.2.1.8。

文档版本	适用产品版本	更新内容	更新日期
2218A01	V2.2.1.8	更新如下内容： 更新版权声明。	2025/9/29
2217P1A01	V2.2.1.7_P1	无	2025/6/13
2217A01	V2.2.1.7	无	2025/4/18
2216P2A01	V2.2.1.6_P2	无	2024/12/6
2216P1A01	V2.2.1.6_P1	无	2024/9/29
2216A01	V2.2.1.6	无	2024/8/8
2215A01	V2.2.1.5	新增如下内容： 2.2.2 验证key是否合法：新增章节。 更新如下内容： 2.2 使用Interaction访问RDS数据：更新描述。	2024/6/28
2214A01	V2.2.1.4	TongRDS V2.2.1.4第一次正式发布。	2023/12/29

前言

本文档是 TongRDS V2.2 产品的用户使用手册之一，详细介绍 TongRDS 提供的开发功能。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

● 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

● 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

● 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握 Linux 系统基本操作；
2. 熟悉 Java 开发语言。

● 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

● 实践经验

为了最大程度地受益于本文档，建议您具备一定的实践经验。这将帮助您更好地应用文档中的操作指南和建议。

请注意，本文档并不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容或所需背景有任何疑问，请在使用之前咨询相关专业人士或寻求额外的支持。

用户使用手册集

TongRDS V2.2 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongRDS_V2.2 用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongRDS_V2.2 快速使用手册	提供产品介绍，安装部署产品的基本步骤及验证方式。
3	002_TongRDS_V2.2 配置参数手册	提供配置文件及参数的详细说明。
4	003_TongRDS_V2.2 管理控制台安装手册	详细介绍管理控制台的安装卸载、启动停止及登录。

编号	手册集文档	说明
5	004_TongRDS_V2.2 管理控制台使用手册	提供管理控制台基础信息、RDS 管理、系统监控、系统审计等功能的使用说明。
6	005_TongRDS_V2.2 客户端使用手册	介绍 TongRDS 提供的客户端程序使用说明。
7	006_TongRDS_V2.2 监控管理手册	提供安装部署监控管理组件的基本步骤。
8	007_TongRDS_V2.2 场景配置参考手册	提供常见部署模式的配置示例及验证方法。
9	008_TongRDS_V2.2 命令&接口使用手册	提供命令及接口的详细说明及使用方法。
10	009_TongRDS_V2.2 开发手册	详细介绍 TongRDS 提供的开发功能。
11	010_TongRDS_V2.2 运维手册	提供 TongRDS 常见问题的解决方案。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

电话：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 硬件及软件信息
5. 操作系统及其版本
6. 产品版本号
7. 日志等错误的详细信息

目录

版本变更说明.....	3
前言.....	4
目录.....	6
第 1 章 概述.....	8
第 2 章 RDS 功能扩展开发.....	9
2.1 实现 PluginModule 接口成为插件	9
2.1.1 getCommands 方法	9
2.1.2 process 方法	9
2.1.3 init 方法	10
2.1.4 getHash 方法	11
2.2 使用 Interaction 访问 RDS 数据	11
2.2.1 访问 RDS 数据.....	11
2.2.2 验证 key 是否合法.....	12
2.2.3 输出日志.....	12
2.3 应用举例	12
2.3.1 数据脱敏.....	12
2.3.2 单 key 存超大量数据.....	15
2.4 开发限制	22
第 3 章 Lua 扩展开发	23
3.1 编写客户化功能模块	23
3.2 加载客户化模块	23
3.3 应用举例	23
3.3.1 科学计算工具.....	23
第 4 章 客户端开发.....	27
4.1 使用 SDK 进行客户端开发	27
4.1.1 直连服务节点.....	27
4.1.2 连接池方式连接服务节点.....	28
4.1.3 通过中心节点连接服务.....	30
4.1.4 命令接口.....	31
4.1.5 程序样例.....	39
4.2 Redis 兼容方式客户端开发.....	39
4.2.1 Redis 兼容性列表	39
4.2.2 测试例程.....	41
4.3 TCP/IP 方式客户端开发.....	41
4.3.1 命令请求.....	41

4.3.2 命令说明.....	42
4.3.3 命令响应.....	46
4.3.4 命令用例.....	46
4.3.5 错误代码表.....	48
4.3.6 命令操作举例.....	49
4.3.7 特殊命令.....	49
4.3.8 客户端 API 演示程序.....	51
附录 A SetSplit2MultiModule.java.....	55
附录 B TestClient.java	65
附录 C TestClientPooled.java.....	68
附录 D TestClientCenter.java.....	69
附录 E 程序样例	70
附录 F JedisTest.java	73
附录 G 术语说明	76

第1章 概述

本文档详细介绍 RDS 提供的开发功能。目前包含 2 部分：

- RDS 针对客户化需求提供的客户化开发接口（插件开发），用于支持不同客户的定制化需求，包括新命令扩展、已有命令的功能扩展等；
- 通过 RDS 提供的 SDK 工具访问 RDS 原生功能的应用开发。

第2章 RDS功能扩展开发

RDS 从 V2.2.1.2 版本开始提供插件扩展功能，该扩展功能独立于 RDS 核心，可自行根据需要定制开发，不影响 RDS 原有产品功能。

插件开发主要用于针对不同客户的特定需求增强 RDS 功能，如：增加新的命令，或增强已有命令的能力。

插件需要实现：

1. 通知 RDS 需要拦截哪些命令；
2. RDS 收到对应命令时触发插件，将处理权交给插件；
3. 插件需要有访问 RDS 内部数据；
4. 插件返回处理结果；
5. 插件需要通过 RDS 的日志系统写出日志（用于调试和跟踪）。

2.1 实现PluginModule接口成为插件

开发插件首先需要实现 PluginModule（com.memdb.util.PluginModule）接口。该接口包含 4 个方法，有 3 个方法必须显式实现，分别是 process、getCommands、config。

2.1.1 getCommands方法

getCommands 方法没有参数，返回一个（小写）字符串格式的命令数组，用于通知 RDS 插件需要拦截的命令列表。注意：程序可以同时加载多个插件，但同一个命令只能被一个插件拦截，如果某个插件加载时命令已经被其他插件拦截则加载失败并报错。该方法实现举例如下：

```
@Override
public String[] getCommands() {
    return new String[]{"hset", "hget", "hdel", "hgetall", "hexists", "hincrby", "hincrbyfloat", "hkeys",
        "hlen",
        "hmget", "hmset", "hsetnx", "hvals", "hscan", "del", "type"};
}
```

2.1.2 process方法

process 方法是命令的实际处理方法，方法定义为：

```
Object process(List in, Interaction interaction, ClientContext context);
```

其中参数定义：

- **in:** 客户端输入的命令，实际是 Vector，其中第一项是字符串形式的敏弓，后面各项均为 byte[] 类型；
- **interaction:** RDS 内部数据访问接口类，该参数是一个 Interaction 接口的实现类，用于提供插件使用标准命令访问 RDS 数据的能力；
- **context:** 当前客户端连接的上下文信息。可通过该对象得到：当前命令对应的 db (int getDb()); 当前客户端序列号 (long getClientId()); 当前客户端名称 (String getClientName()); 当前客户端地址 (String getClientAddress()) 和端口 (int getClientPort()); 当前客户端连接 (ChannelHandlerContext getChannel()); 设置当前客户连接上下文的附加属性 (void setProperty(String key,Object value)); 读取当前客户连接上下文的附加属性 (Object getProperty(String key))。

返回值 Object 根据不同类型有不同含义：

- **Boolean:** 返回的是执行成功（对应协议接口返回的“+OK”）；
- **Long:** 返回的是整数；
- **Double:** 返回的是浮点数（RESP3 支持，RESP2 时浮点数将返回 bulk 字符串）；
- **String:** 返回的是简单字符串；
- **byte[]:** 返回的是 bulk 字符串；
- **null:** 返回的是空的 bulk 字符串（对应接口返回的“\$-1”）；
- **List:** 返回的是数组（数组可以嵌套）；
- **Map:** 返回的是 map（RESP3 协议支持 map）；
- **Exception:** 返回的是执行失败，getMessage()方法返回失败原因。

2.1.3 init方法

init 方法用于初始化插件，当插件被实例化后会立刻调用该方法，并将配置文件中针对插件的属性配置传递给插件，提供插件通过标准的 cfg.xml 配置文件配置插件的能力。插件可在此方法内完成自身初始化工作。该方法的参数是一个 Properties 实例，实例中以键值对的方式保存 cfg.xml 针对该插件的属性配置。例如 cfg.xml 中定义如下插件：

```
<Plugins>

  <Class suffix="^_<sup>^</sup>">

com.memdb.modules.HashSplit2MultiModule

  </Class>

</Plugins>
```

当程序初始化 HashSplit2MultiModule 实例时会调用该实例的 config 方法，并在传入的 properties 参数中包含（key=suffix，value=^_^）的项。

2.1.4 getHash方法

getHash 方法由接口 PluginModule 提供了缺省实现，该方法本质上是一个定制化的为 byte 数组计算 hashCode 的方法（返回值保证为正数）。附带的 max 参数用于限制返回的 hashCode 的最大值，例如输入 2 则返回值只有 0 或 1，输入 0 则不限制最大值。

2.2 使用Interaction访问RDS数据

开发插件就是为了对原有 RDS 数据进行深度加工满足定制化的需要，其最基本的需求是要能访问到 RDS 内部的数据。上述 process 方法中定义的参数 interaction 提供访问 RDS 的能力。该参数必不为空，且实现了 com.memdb.util.Interaction 接口。Interaction 接口提供访问 RDS 数据的 call 方法、验证 key 是否在当前节点的 checkValid 方法、写日志的 log 方法，以及获取当前日志级别的 logLevel 方法。

2.2.1 访问RDS数据

使用 Interaction 接口的 call 方法（Object call(List data, ClientContext context)）访问 RDS 数据。call 方法的输入参数为一个 list 和 context，context 是当前执行命令的上下文，从 PluginModule 实现的 process 方法调用中传递（call 方法只在 process 中调用）；list 是访问 RDS 的操作命令描述，list 的内容定义为：第一个字段必须是小写字串形式的命令名（如：“get”），后面字段建议均采用 byte[] 字节数组形式提供，内容为命令所需参数。

注意：

1. list 参数不能为空，且 list 中至少有 1 项内容；
2. list 中的第一项必须是字符串（String）类型；
3. 剩余项必须是字节数组（byte[]）类型。

否则可能会引起不可预知的错误。

返回值根据类型不同表示不同数据。与上述 1.2 中定义的 process 方法的返回参数相同。

例如需要使用 type 命令查 key 为 “aaa” 的数据的类型：

```
ArrayList list = new ArrayList<>();
list.add("type");
list.add("aaa".getBytes());
String resp = (String) interaction.call(list, db);
```

```
If (instruction.logLevel() == Interaction.LOGLEVEL.INFO) {
    interaction.log(Interaction.LOGLEVEL.INFO, "type is " + resp);
}
```

2.2.2 验证key是否合法

使用 Interaction 接口的 checkValid 方法(String command,boolean isWrite, byte[]... keys)同时验证多个 key 是否合法（目前程序的验证逻辑只在集群模式下验证了 key 是否在当前节点）。

由于扩展接口可以开发任意命令和功能，因此 RDS 无法像标准命令那样获得 key，就需要开发者在插件中调用此方法验证，否则可能该功能在集群下工作不正常。

该方法至少需要 3 个参数，第一个参数是小写字符串形式的命令，第二个参数指定该命令是否会改变数据内容（即是否为写命令），后续为需要验证的 key。

2.2.3 输出日志

通常插件需要通过日志跟踪调试、排查运行错误。Interaction 的 log 方法用于利用 RDS 的日志系统输出日志信息。

Interaction 的 logLevel 方法用于查询当前日志系统的输出级别。

2.3 应用举例

2.3.1 数据脱敏

2.3.1.1 需求

应用使用 RDS 的 hash 数据保存用户信息，每个用户一条记录，用户 id 是 key，value 是用户信息内容。要求应用存入电话号码字段（可能有多个字段，如：本人电话、紧急联系人电话等）时自动将数据脱敏保存，脱敏规则为保留号码前 3 位，随后内容用星号 “*” 替代。

2.3.1.2 实现思路

应用要求 RDS 中保存的数据已经是脱敏后的，因此需要在写入数据时自动脱敏，hash 数据的写入命令有两个：hget 和 hmget，插件需要拦截这 2 个命令对输入数据进行脱敏处理然后再调用 RDS 的标准命令写入。

数据读取时直接返回脱敏数据即可，不需要做特殊处理。因此本插件只需要拦截 hget 和 hmget 这 2 个命令。

2.3.1.3 具体实现

首先创建一个类 `AntiSensitive` 并实现 `PluginModule` 接口。

实现接口的 `init` 方法，读取配置参数并初始化插件。本插件在配置文件中的 `sensitive-field` 属性配置了需要脱敏的 field 名：

```
@Override

public void init(Properties properties) {

    HashSet<String> sens = new HashSet<>();

    String names = properties.getProperty("sensitive-field");

    if (names != null) {

        String[] name = names.trim().split("[\\t]*,[\\t]*");

        for (String n : name) {

            if (n != null && n.length() > 0) {

                sens.add(n);

            }

        }

    }

    Sensitived = sens;

}
```

实现接口的 `getCommands` 方法，本插件只需要拦截 2 个命令，方法实现如下：

```
@Override

public String[] getCommands() {

    // 本插件拦截 hset 和 hmset 命令

    return new String[]{"hset", "hmset"};

}
```

实现接口的 `process` 方法，当 RDS 收到前述方法指定的命令时会调用插件的该方法：

```
@Override

public Object process(List list, Interaction interaction, int db) {

    // 只有 hset 命令和 hmset 命令会调用本方法

    // 换言之，list.get(0)只可能是“hset”或“hmset”

    // list.get(1)是 key，类型为 byte[]

    // list.get(2)开始是 field 和 value 对

    for (int i = 2; i < list.size() + 1; i += 2) {

        anti(list, i);

    }

    return interaction.call(list, db);

}
```

实现一个脱敏方法 anti:

```
private void anti(List data, int idx) {

    if (data == null || data.size() <= idx + 1) {

        return;

    }

    String name = new String((byte[]) data.get(idx), StandardCharsets.UTF_8);

    if (Sensitived.contains(name)) {

        byte[] v = (byte[]) data.get(idx + 1);

        for (int i = 3; i < v.length; ++i) {

            v[i] = '*';

        }

    }

}
```

2.3.1.4 插件源码参考

插件源码参见示例中的“rds-plugin”工程。

2.3.1.5 插件部署

插件开发独立于 RDS 标准功能，需要部署后才能生效。部署插件步骤：

1. 将开发的插件的 jar 包复制到已经安装好的 pmemdb 的 lib 目录下
2. 修改 cfg.xml 文件在 Server 根标签下加入 Plugins 标签通知 rds 加载指定插件，并指定加载插件需要的配置信息。如果需要同时生效多个插件，可以在 Plugins 下面加入多条 Class，插件将按照配置顺序依次生效，如果同一个命令被多个插件拦截，拦截过程也会依次传递。RDS 日志中可以看到插件的加载状态（成功或失败）。例如本例的配置如下：

```
<Server>
  <Plugins>
    <Class sensitive-field="phone, telephone">
      rds.modules.AntiSensitive
    </Class>
  </Plugins>
</Server>
```

3. 重启 rds 生效。

2.3.2 单key存超大量数据

2.3.2.1 需求

客户的特殊需求：用 set 类型作为大量数据的判重，因为有多多个客户端从不同渠道获取数据，因此需要 RDS 做统一管理。

客户端会将收集到的数据统一写到同一个 key 里面，通过 set 数据特性过滤重复数据。但因为数据量过大（大约有 1.5 亿条），而 RDS 中单个 key 的容量有限无法在同一个 key 中处理这么大量的数据（Redis 单 key 最大为 512M，实测只能存 500 万左右，远远不能满足需要；RDS 单 key 最大为 1.6G，也无法满足）。

2.3.2.2 实现思路

1 个 key 放不下，那么可以把这些数据放到有限的几个 key（比如 32 个 key）中。

可以拦截 set 的插入（sadd）命令，在插入时将 field 分配到不同的 key 里面，需要注意的是多个 key 功能存放数据需要满足单一 key 时的唯一性要求，即 field 不重复。这个可以在操作时根据 field 的内容不同确定（如根据 field 的 hashCode 分配）保存到哪个 key 中。

为了实现此功能又不破坏 RDS 标准功能的通用性，就需要用到本节的插件功能。插件将拦截 set 操作命令并做预处理后再将数据实际写入 RDS。

当通过配置文件加载该插件后就具有了“单个”key 保存超大容量数据的能力，去掉该插件 RDS 则恢复标准功能。

2.3.2.3 具体实现

首先创建一个类并实现 PluginModule 接口。

实现接口方法（init），init 方法只在插件实例化时被首先调用，传入的参数包含 cfg.xml 中配置的插件的属性内容。内容如下：

```
@Override
public void init(Properties properties) {

    String value;

    value = properties.getProperty("suffix");

    if (value != null && value.length() > 0) {

        this.separator = value;

    }

    value = properties.getProperty("blocks");

    if (value != null) {

        try {

            BLOCKS = Integer.parseInt(value);

            if (BLOCKS <= 1) {

                BLOCKS = 2;

            }

        } catch (Throwable t) {

            BLOCKS = DEFAULT_BLOCKS;

        }

    }

}
```



```

    }

    if ("true".equalsIgnoreCase(properties.getProperty("packedblocknumber"))) {
        // 此配置下理论上最多能存储的项是 2 的 32 次方减一。
        // 但采用此配置时 sscan 返回 cursor 值的范围的兼容性更好
        // 该配置只影响 sscan 命令

        int bit = 0;

        while ((1 << bit) < BLOCKS) {

            bit++;

        }

        blockBitPosition = 32 - bit;

        dataMask = (11 << blockBitPosition) - 1;

    }

    value = properties.getProperty("maxItems");

    if (value != null) {

        try {

            maxItems = Integer.parseInt(value);

        } catch (Throwable t) {

            maxItems = DEFAULT_MAXITEMS;

        }

    }

}

```

实现接口方法（getCommands），该方法通知 RDS 本插件拦截的命令：

```

@Override

public String[] getCommands() {

    return new String[]{"sadd", "scard", "sdiff", "sdiffstore", "sinter", "sinterstore", "sunion",
        "sunionstore"};

}

```

```
        , "sismember", "smembers", "smove", "spop", "srandmember", "srem", "sscan", "del",
        "type"};
    }
}
```

实现接口的方法（process）：

```
@Override
public Object process(List in, Interaction interaction, int db) {
    Object ret = null;
    String cmd = (String) in.get(0);
    interaction.log(Interaction.LOGLEVEL.DEBUG, "receive '" + cmd + "'");
    if ("type".equals(cmd)) {
        return getType((byte[]) in.get(1), interaction, db);
    } else if ("del".equals(cmd)) {
        .....

        interaction.log(Interaction.LOGLEVEL.DEBUG, "request: " + cmd + ", response: " + ret);
        return ret;
    }
}
```

2.3.2.3.1 实现 sadd 命令

Sadd 命令是 set 操作的基本命令，该命令可以一次插入多个 field。实现思路位：首先创建一个 list 数组，数组大小为保存该条记录需要的 key 的个数，然后分别将后续的 filed 放到对应的 list 中，然后调用 interaction.call 方法保存到 RDS 中的多个 key 中。代码如下：

```
} else if ("sadd".equals(cmd)) {
    // 多目命令
    long len = 0;
    byte[] key = (byte[]) in.get(1);
    ArrayList[] reqs = new ArrayList[BLOCKS];
```

```

for (int i = 0; i < BLOCKS; ++i) {

    // 创建 key 的数组

    reqs[i] = new ArrayList();

    reqs[i].add(cmd);

    reqs[i].add(getNewKey(key, i));

}

for (int i = 2; i < in.size(); ++i) {

    int block = getHash((byte[]) in.get(i), BLOCKS);

    reqs[block].add(in.get(i));

}

for (int i = 0; i < BLOCKS; ++i) {

    if (reqs[i].size() > 2) {

        Object o = interaction.call(reqs[i], db);

        if (o instanceof Long) {

            len += ((Long) o).longValue();

        } else {

            return o;

        }

    }

}

ret = new Long(len);

```

2.3.2.3.2 实现 srem 方法

Srem 用于删除 key 中指定的多个 field，实现的关键是根据 field 定位到对应的 key。实现代码如下：

```

} else if ("srem".equals(cmd)) {

    // 多目命令

    byte[] key = (byte[]) in.get(1);

    ArrayList[] reqs = new ArrayList[BLOCKS];

```

```

        for (int i = 0; i < BLOCKS; ++i) {

            reqs[i] = new ArrayList();

            reqs[i].add(cmd);

            reqs[i].add(getNewKey(key, i));

        }

        for (int i = 2; i < in.size(); ++i) {

            reqs[getHash((byte[]) in.get(i), BLOCKS)].add(in.get(i));

        }

        long deled = 0;

        for (int i = 0; i < BLOCKS; ++i) {

            if (reqs[i].size() > 2) {

                try {

                    deled += (Long) interaction.call(reqs[i], db);

                } catch (Throwable t) {

                }

            }

        }

        ret = new Long(deled);
    
```

2.3.2.3.3 实现 scard 方法

Scard 是读取 set 内 field 数量的命令，实现关键是根据指定的原始 key，遍历全部相关的 key 然后将各个 key 中保存的数量求和返回，代码如下：

```

    } else if ("scard".equals(cmd)) {

        // 单目命令

        byte[] key = (byte[]) in.get(1);

        long len = 0;

        ArrayList req = new ArrayList();

        req.add(cmd);
    
```

```

req.add("");

for (int i = 0; i < BLOCKS; ++i) {

    req.set(1, getNewKey(key, i));

    Object o = interaction.call(req, db);

    if (o instanceof Long) {

        len += (Long) o;

    } else if (o instanceof Exception) {

        return o;

    }

}

ret = new Long(len);

```

2.3.2.3.4 实现 sismember 命令

Sismember 命令实现比较简单，只要找到对应的 key，看看有没有指定的 field 即可。代码如下：

```

} else if ("sismember".equals(cmd)) {

    // 双目命令

    byte[] key = (byte[]) in.get(1);

    byte[] field = (byte[]) in.get(2);

    ArrayList req = new ArrayList();

    req.add(cmd);

    req.add(getNewKey(key, field));

    req.add(field);

    ret = interaction.call(req, db);
}

```

2.3.2.3.5 其他命令

Set 的其他命令实现原理完全一样，此处不再赘述，感兴趣可以参考 SetSplit2MultiModule 源码。

另外需要说明的就是拦截并重新实现了 set 命令以后，其他相关命令（如：type、del 等）也需要一并拦截处理。

2.3.2.4 编译打包插件

插件开发完成后独立编译打包，为了避免插件之间依赖包的版本冲突，要求插件编译后必须和所需的依赖包一起打包成 1 个 jar 文件，并且迁移其中依赖包的包名为私有包名（使用 maven 的 maven-shade-plugin 插件，或 gradle 的 com.github.johnrengelman.shadow 插件实现）。

2.3.2.5 插件源码参考

参考附录 A SetSplit2MultiModule.java。

2.3.2.6 插件部署

插件开发独立于 RDS 标准功能，需要部署后才能生效。部署插件步骤：

1. 将开发的插件的 jar 包复制到已经安装好的 pmemdb 的 lib 目录下
2. 修改 cfg.xml 文件在 Server 根标签下加入 Plugins 标签通知 rds 加载指定插件，并指定加载插件需要的配置信息。如果需要同时生效多个插件，可以在 Plugins 下面加入多条 Class，插件将按照配置顺序依次生效，如果同一个命令被多个插件拦截，拦截过程也会依次传递。RDS 日志中可以看到插件的加载状态（成功或失败）。例如生效上节开发的 SetSplit2MultiModule 插件的配置如下：

```
<Server>

  <Plugins>

    <Class suffix="^_ ^" blocks="32" maxItems="10000" packedblocknumber="true">

      com.memdb.modules.SetSplit2MultiModule

    </Class>

  </Plugins>

  .....

</Server>
```

3. 重启 rds 生效。

2.4 开发限制

RDS 需要运行时动态加载插件，因此需要插件提供缺省构造方法，否则无法加载成功。

第3章 Lua扩展开发

Redis 协议支持执行 lua 脚本，RDS 支持针对 lua 脚本的能力扩展，将客户化开发的模块加载到 RDS 的 Lua 环境，可在 lua 脚本中调用增强 lua 脚本的能力。开发大致需要以下步骤：

1. 编写客户化功能模块（主类必须是具有缺省构造方法的 java 类）；
2. 将开发完成的类通过配置加载到 RDS 中；
3. 编写 lua 脚本调用。

3.1 编写客户化功能模块

根据实际功能需要编写功能模块，针对开发没有其他限制，只需要模块的主类（需要直接被 RDS 加载的类）提供缺省参数的构造方法即可。

3.2 加载客户化模块

修改服务节点的 cfg.xml 中的 LuaObjects 节用于配置需要加载的 lua 客户化模块，配置规则为脚本中调用的对象名对应实际 java 类名，同时可配置模块运行所需的参数。例如如下配置：

```
<LuaObjects>

    <SC pi="3.14159">

        com.server.objects.Scientific

    </SC>

</LuaObjects>
```

定义了一个名为 SC 的 lua 中的对象，该对象对应的实际类型为 com.server.objects.Scientific，并为该对象配置了一个参数 pi。

在 com.server.objects.Scientific 中可以通过调用 System.getProperty("com.server.objects.Scientific-pi")得到该配置的值（key 的规则为 classname-propertyname）。

3.3 应用举例

3.3.1 科学计算工具

本例用于演示 lua 客户化扩展开发部署的过程和效果。

3.3.1.1 用例说明

本例子实现了一个 add 加法函数和一个 angle2Radian 弧度转换函数，其中 add 函数演示了如何在 java 程序中返回给 lua 多个返回值；angle2Radian 函数演示了如何读取并使用配置的参数。

3.3.1.2 程序源码

```
package com.server.objects;

/**
 * config in cfg.xml:
 *
 *      <LuaObjects>
 *          <SC pi="3.14159">
 *              com.server.objects.Scientific
 *          </SC>
 *      </LuaObjects>
 *
 */
public class Scientific {
    /**
     * 演示参数调用和多返回值
     *
     * eval "local i,j = SC:add(1,2);return i;"
     * :3
     * eval "local i,j = SC:add(1,2);return j"
     * $2
     * OK
     *
     * @param i 被加数
     */
}
```



```

* @param j 加数

* @return 运算结果和文字说明，对应 lua 中的 2 个返回值（一个整数，一个字符串）

*/

public Object[] add(int i, int j) {

    return new Object[] {i + j, "OK"};

}

/**

* 演示从配置文件中读参数

*

* eval "return SC:angle2Radian(30)"

* $8

* 9.549305

*

* @param angle 角度

* @return 弧度

*/

public double angle2Radian(double angle) {

    String cfg_pi = System.getProperty("com.server.objects.Scientific-pi");

    double pi = Math.PI;

    if (cfg_pi != null) {

        try {

            pi = Double.parseDouble(cfg_pi);

        } catch (Throwable t) {

        }

    }

    return angle / pi;

}

```

```
}
```

3.3.1.3 配置

```
<LuaObjects>

  <!-- lua 功能模块名为 SC，实现类为 com.server.objects.Scientific -->

  <!-- 可以定义多个属性值作为对象的配置使用，如下配置的属性 pi -->

  <!-- 属性 pi 在 Scientific 类中通过 System.getProperty("com.server.objects.Scientific-pi") 调用获得 -->

  <!-- System.getProperty 调用的 key 为：类名 + "-" + 属性名，返回配置的属性值 -->

  <!-- 类中方法的返回值必须是 Object 或其子类，lua 会根据返回类型转换为 lua 对象 -->

  <SC pi="3.14159">

    com.server.objects.Scientific

  </SC>
```

3.3.1.4 功能测试

使用 Client.sh 工具连接 RDS 服务，输入 `eval "return SC:angle2Radian(30)"` 观察返回结果如下验证模块加载成功：

```
ding@DESKTOP-ILUMUAP:/mnt/e/pmemdb/bin$ client.sh -r -p 6379 -a 123
Redis protocol enabled.
Node connection.

client is connected to localhost : 6379
localhost:6379 > eval "return SC:angle2Radian(30)"
localhost:6379 > "9.549305"
localhost:6379 > █
```

第4章 客户端开发

4.1 使用SDK进行客户端开发

为了方便客户端应用的开发，RDS 提供了客户端 SDK 包（目前可提供 java 版本）。客户端 SDK 包对底层 Socket 连接、连接保持和异常切换、连接池、服务节点变更通知的订阅和处理等逻辑进行了封装，方便客户端开发。

RDS 现已提供了 Redis 兼容模式，采用兼容模式（兼容模式下 RDS 可支持 Redis 的通用功能），已经使用了 Redis 的应用程序可以保持不变直接更换 Redis，充分保护原有投资，并利用 Redis 丰富的、多种语言版本的客户端程序。

RDS 企业版应用中间件包含 2 类服务：数据服务节点和中心管理节点。数据服务节点（以下简称“服务节点”）提供实际数据缓存服务功能，包括数据存取、计算等；中心管理节点（以下简称“中心节点”）提供对服务节点的监控和管理，帮助客户端及时掌握服务节点的状态，发现集群的节点变化，更有效可靠的实现缓存服务。

RDS 服务节点采用标准 TCP/IP 方式通信，命令请求与应答采用行方式完成，即每个命令请求为 1 行，以换行符结束；应答也采用同样的行方式定义。（测试可采用 telnet 方式直接连接服务端口）。

SDK 包提供直接连接服务节点和连接中心节点两种接入方式，支持创建单一连接和创建连接池的管理方式。采用 SDK 包连接 RDS 服务器，首先在客户端工程中引入文件“javaclient-1.0.jar”（其中 1.0 为版本号，后期会随着版本升级更新）。然后在程序中创建 RDS 服务连接。

4.1.1 直连服务节点

1. 创建连接管理对象：

```
import com.rds.client.javaclient.Client;

Client client = new Client(false, false, null
    , new InetSocketAddress("192.168.0.60", 6200)
    , new InetSocketAddress("192.168.0.60", 6201));
```

其中，前 3 个参数分别是：是否采用 ssl、是否需要密码认证、密码。这 3 个参数需要和服务节点的配置对应。密码后面跟 1 个或多个服务节点的地址（高可用需要）。

2. 得到连接对象：

利用上面创建的 Client 获得连接对象：

```
Connection connection=client.getConnection();
```

3. 访问服务：

Connection 对象封装了访问数据节点的各项功能方法，如 set、get 等。

4. 程序举例：

```
import com.rds.client.javaclient.Client;

import com.rds.client.javaclient.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient1 {
    public static void main(String[] arg) throws IOException {
        Client client = new Client(false, false, null
            , new InetSocketAddress("192.168.0.80", 6200)
            , new InetSocketAddress("192.168.0.81", 6200));

        Connection connection = client.getConnection();

        connection.set("str-key1", "str-value1");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}
```

样例程序请参考附录 B TestClient.java。

4.1.2 连接池方式连接服务节点

使用连接池可以方便客户端程序复用已经创建的连接，避免频繁创建、关闭连接。采用连接池和上述直接连接方式类似，唯一的区别是将 Client 对象换成 ConnectionPool 对象，其它用法完全一致。

注意：采用连接池方式，Connection 对象使用完后一定要调用其 close 方法，否则会很快占满连接池。

程序参考如下：

```
import com.rds.client.javaclient.PooledClient;

import com.rds.client.javaclient.Connection;

import java.io.IOException;
```

```
import java.net.InetSocketAddress;

public class TestClientPooled {

    public static void main(String[] arg) throws IOException {

        // ConnectionPool(boolean useSsl, boolean needPwd, String password, InetSocketAddress... addrs)

        PooledClient pool = new PooledClient(false, false, null

            , new InetSocketAddress("192.168.0.60", 6200)

            , new InetSocketAddress("192.168.0.60", 6201));

        Connection connection = pool.getConnection();

        connection.set("str-key1", "str-value1");

        connection.set("str-key2", "str-value2");

        System.out.println("value = " + connection.get("str-key1"));

        connection.close();

        connection = pool.getConnection();

        System.out.println("value = " + connection.get("str-key2"));

        connection.close();

        pool.close();

    }

}
```

样例程序请参考附录 C [TestClientPooled.java](#)。

4.1.3 通过中心节点连接服务

对于企业版用户在有部署中心节点的 RDS 现场，客户端程序连接中心节点会得到更可靠和更及时的服务保障，当连接中心节点后，客户端可获得对应服务集群的服务节点信息、节点变更等信息，能更可靠的获得服务保障。

1. 创建连接管理对象：

```
import com.rds.client.javaclient.CenterClient;

CenterClient client = new CenterClient("WebSession","acioweor_483kja03np4h8238G"
    , new InetSocketAddress("192.168.0.60", 6300)
    , new InetSocketAddress("192.168.0.60", 6301));
```

其中，前 2 个参数分别是：客户端使用的服务名、客户端授权串。中心节点通过这 2 个参数对客户端请求进行认证和授权（未授权客户端无法接入）。授权码后面跟 1 个或多个中心节点的地址（不再需要配置服务节点的信息，相关信息会从中心节点获得）。

2. 得到连接对象：

利用上面创建的 Client 获得连接对象：

```
Connection connection=client.getConnection();
```

3. 访问服务：

Connection 对象封装了访问数据节点的各项功能方法，如 set、get 等。

4. 程序举例：

```
import com.rds.client.javaclient.CenterClient;

import com.rds.client.javaclient.Connection;

import java.io.IOException;

import java.net.InetSocketAddress;

public class TestClientCenter {

    public static void main(String[] arg) throws IOException {

        CenterClient client = new CenterClient("WebSession", "acioweor_483kja03np4h8238G"
```

```

        , new InetSocketAddress("192.168.0.60", 6300)
        , new InetSocketAddress("192.168.0.60", 6301));

    Connection connection = client.getConnection();

    connection.set("str-key1", "str-value1");

    System.out.println("value = " + connection.get("str-key1"));

    connection.close();
}
}

```

样例程序请参考附录 D [TestClientCenter.java](#)。

4.1.4 命令接口

Connection 类封装了访问 RDS 各功能的操作方法，方法列表如下：

4.1.4.1 append

在已有字符串基础上追加内容，如果不存在则创建新字符串。

4.1.4.2 decr

将 key 指定的 value 当作整数，减去一个给定值（缺省减 1）。

4.1.4.3 del

删除指定 key 的记录。

4.1.4.4 dels

根据索引内容删除所有匹配的 key，支持 1 到 3 个索引联合删除。

4.1.4.5 exists

判断指定 key 是否存在（且未过期）。

4.1.4.6 expire

设置指定 key 的相对过期时间，单位是秒。

4.1.4.7 expireat

设置指定 key 的绝对过期时间，单位秒。

4.1.4.8 get

根据输入 key，获得指定字符串类型的 value 内容。

4.1.4.9 gets

根据索引查全部符合的字符串记录的 value 内容。

4.1.4.10 getset

获得指定 key 的内容并用新内容覆盖。

4.1.4.11 hexists

判断指定的 hash 类型的数据是否包含给定项。

4.1.4.12 hget

获得 hash 类型数据中的指定项内容。

4.1.4.13 hgetAll

返回指定 hash 类型数据的全部项。

4.1.4.14 hincr

指定 hash 类型数据中的对应项的内容增加整数值。

4.1.4.15 hincrByFloat

指定 hash 类型数据中的对应项的内容增加浮点数值。

4.1.4.16 hkeys

返回指定 key 对应的 hash 类型数据包含的全部 key。

4.1.4.17 hlen

返回指定 key 对应的 hash 类型数据包含的项数。

4.1.4.18 hmget

获得 key 指定的 hash 类型数据中的多个项的内容。

4.1.4.19 hset

设置 key 指定的 hash 类型数据的指定项，如果指定项不存在则新建，已存在则覆盖。

4.1.4.20 hsetnx

设置 key 指定的 hash 类型数据的指定项，如果该项已存在则执行失败。

4.1.4.21 hvals

返回指定 key 对应的 hash 类型数据包含的全部 value 内容。

4.1.4.22 incr

将 key 对应的 string 类型的数当作整数，增加对应的值（缺省加 1）。

4.1.4.23 incrByFloat

将 key 对应的 string 类型的数当作浮点数，增加对应的值。

4.1.4.24 llen

返回 key 对应的 list 类型数据的列表长度。

4.1.4.25 lpop

将指定 list 数据弹出头部的项。

4.1.4.26 lpush

由 key 指定的 list 数据从 list 头部加入 1 项。

4.1.4.27 lpushx

将一个值插入到已存在的列表头部，列表不存在时操作无效。

4.1.4.28 lrange

返回指定范围的列表项。

4.1.4.29 lset

设置指定位置的列表项，如果指定的索引位置超过 list 总长度则执行失败。

4.1.4.30 ltrim

按照输入参数截取 list。

4.1.4.31 mget

获得多个 key 对应的 string 类型数据的 value 值。

4.1.4.32 persist

移除给定 key 的过期时间，使得 key 永不过期。

4.1.4.33 pexpire

设置 key 的相对过期时间，单位毫秒。

4.1.4.34 pexpireAt

设置 key 的绝对过期时间，单位毫秒。

4.1.4.35 psetex

设置 key-value 值，并以毫秒为单位设置 key 的生存时间，单位毫秒。

4.1.4.36 pttl

返回指定 key 多长时间后过期，单位毫秒。

4.1.4.37 rpop

弹出 list 列表尾的项。

4.1.4.38 rpush

项 list 项的尾加入项。

4.1.4.39 rpushx

将一个值插入到已存在的列表尾部(最右边)。如果列表不存在，操作无效。

4.1.4.40 sadd

将成员元素加入到集合中，已经存在于集合的成员元素将被忽略。

4.1.4.41 scard

返回集合中元素的数量。

4.1.4.42 sdiff

返回给定集合之间的差集。不存在的集合 key 将视为空集。

4.1.4.43 sdiffstore

将给定集合之间的差集存储在指定的集合中。如果指定的集合 key 已存在，则会被覆盖。

4.1.4.44 select

设置当前在用的表 id，对应服务节点的 cfg.xml 中的配置，缺省是 1。

4.1.4.45 set

设置指定 key 的字符串的 value 值。

4.1.4.46 setex

设置 key-value 值，并以秒为单位设置 key 的生存时间，单位秒。

4.1.4.47 setnx

设置指定 key 的字符串的 value 值，如果 key 已经存在则操作失败。

4.1.4.48 sinter

返回给定所有给定集合的交集。不存在的集合 key 被视为空集。当给定集合当中有一个空集时，结果也为空集（根据集合运算定律）。

4.1.4.49 sinterStore

将给定集合之间的交集存储在指定的集合中。如果指定的集合已经存在，则将其覆盖。

4.1.4.50 sismember

判断成员元素是否是集合的成员。

4.1.4.51 smembers

返回集合中的所有成员。不存在的集合 key 被视为空集合。

4.1.4.52 spop

移除集合中的指定 key 的最后一个元素，移除后会返回移除的元素。

4.1.4.53 srem

移除集合中的指定成员元素，不存在的成员元素会被忽略。

4.1.4.54 strlen

返回 key 指定的字符串的字符数。

4.1.4.55 substr

截取 key 指定的字符串的指定部分，超出范围则返回错误。

4.1.4.56 sunion

返回给定集合的并集。不存在的集合 `key` 被视为空集。

4.1.4.57 sunionStore

将给定集合的并集存储在指定的集合 `destination` 中。如果 `destination` 已经存在，则将其覆盖。

4.1.4.58 ttl

返回指定 `key` 多长时间后过期，单位秒。

4.1.4.59 upd

更新指定 `key` 的记录，`key` 不存在则操作失败。

4.1.4.60 Zadd

有序集合向列表中添加成员。

4.1.4.61 Zcard

有序集合查列表总长度。

4.1.4.62 Zcount

有序集合计算指定分值区间内的成员数。

4.1.4.63 Zincrby

有序集合改变现有成员的分值。

4.1.4.64 Zinterstore

有序集合求 2 列表的交集并存到新列表中。

4.1.4.65 Zlexcount

有序集合根据 `key` 的范围计算区间内的成员数。

4.1.4.66 Zrange

有序集合返回指定下标区间内的成员。

4.1.4.67 Zrangebylex

有序集合返回 key 指定的区间范围内的成员数。

4.1.4.68 Zrangebyscore

有序集合返回指定分值范围内的成员数。

4.1.4.69 Zrank

有序集合返回指定成员的索引。

4.1.4.70 Zremrangebyscore

有序集合删除分值（score）指定的区间范围内的成员。

4.1.4.71 Zremrangebyrank

有序集合删除下标指定的区间范围内的成员。

4.1.4.72 Zrevrange

有序集合逆序（分值由高到低）返回指定下标区间内的成员。

4.1.4.73 Zrevrangebyscore

有序集合逆序（分值由高到低）有序集合返回指定分值范围内的成员数。

4.1.4.74 Zrevrank

有序集合逆序（分值由高到低）返回指定成员的索引。

4.1.4.75 Zscore

有序集合返回指定成员的分值。

4.1.4.76 Zunionstore

有序集合求 2 列表的并集并存到新列表中。

4.1.5 程序样例

几种连接 RDS 的程序样例请参考[附录 E 程序样例](#)。

4.2 Redis兼容方式客户端开发

为了增加 RDS 的可用性，增加了 Redis 的兼容接口，并实现了 Redis 常用功能，对外接口保持和 Redis 一致，使得已有应用可以不修改程序代码无侵入的更换为 RDS，甚至可以利用 Redis 现有的多种语言版本的客户端程序来使用 RDS 服务。

同时，针对 Redis 安全缺陷，在敏感信息处理等方面进行了加强，增加了接入端认证和授权，一定程度提高了系统的安全性。

4.2.1 Redis兼容性列表

目前 RDS 已经实现了 Redis 常用功能，下表是已经支持、由 jedis 客户端测试通过的功能列表：

缓存中间件兼容 Redis 功能列表			
服务器命令（17 个）			
PING	AUTH	SELECT	QUIT
INFO	DBSIZE	SAVE	BGSAVE
LASTSAVE	COMMAND	ECHO	TIME
FLUSHDB	FLUSHALL	WAIT	READONLY
ROLE	-	-	-
KEY 操作类命令（17 个）			
EXISTS	EXPIRE	PEXPIRE	EXPIREAT
PEXPIREAT	PERSIST	TTL	PTTL
DEL	KEYS	SCAN	TYPE
UNLINK	DUMP	RESTORE	SORT
SORT_RO	-	-	-
STRING 操作类（32 个）			
SET	SETEX	SETNX	PSETEX
GET	GETSET	APPEND	SUBSTR
INCR	INCRBY	INCRBYFLOAT	DECR
DECRBY	STRLEN	GETRANGE	MGET
RENAME	RENAMENX	MSET	MSETNX
GETBIT	SETBIT	BITFIELD	BITFIELD_RO
BITPOS	BITCOUNT	BITOP	SETRANGE

缓存中间件兼容 Redis 功能列表			
PFADD	PFCOUNT	PFMERGE	GETEX
LIST 操作类（17 个）			
LPUSH	LPOP	LPUSHX	LINDEX
RPUSH	RPOP	RPUSHX	LSET
LRANGE	LLEN	LTRIM	LREM
BLPOP	BRPOP	LINSERT	RPOPLPUSH
BRPOPLPUSH	-	-	-
SET 操作类（15 个）			
SADD	SMEMBERS	SISMEMBER	SREM
SPOP	SCARD	SDIFF	SDIFFSTORE
SINTER	SINTERSTORE	SUNION	SUNIONSTORE
SSCAN	SMOVE	SRANDMEMBER	-
HASH 操作类（13 个）			
HSET	HMSET	HGET	HGETALL
HEXISTS	HKEYS	HVALS	HINCRBY
HINCRBYFLOAT	HSETNX	HLEN	HSTRLEN
HSCAN	-	-	-
ZSET 操作类（27 个）			
ZADD	ZCARD	ZCOUNT	ZINCRBY
ZINTERSTORE	ZLEXCOUNT	ZRANGE	ZRANGEBYLEX
ZRANGEBYSCORE	ZRANK	ZREM	ZREMRANGEBYLEX
ZREMRANGEBYRANK	ZREMRANGEBYSCORE	ZREVRANGE	ZREVRANGEBYSCORE
ZREVRANK	ZSCORE	ZUNIONSTORE	ZSCAN
ZPOPMIN	ZPOPMAX	BZPOPMIN	BZPOPMAX
ZINTER	ZUNION	ZREVRANGEBYLEX	-
GEO 操作类（6 个）			
GEOADD	GEODIST	GEOPOS	GEOHASH
GEORADIUS	GEORADIUSBYMEMBER	-	-
STREAM 操作类（13 个）			
XADD	XLEN	XDEL	XREAD
XRANGE	XREVRANGE	XACK	XTRIM
XCLAIM	XGROUP	XREADGROUP	XINFO
XPENDING	-	-	-
LUA 脚本操作（6 个）			
EVAL	EVALSHA	SCRIPT EXISTS	SCRIPT FLUSH
SCRIPT LOAD	SCRIPT KILL	-	-
Redis 发布订阅（6 个）			

缓存中间件兼容 Redis 功能列表			
PUBLISH	PUNSUBSCRIBE	UNSUBSCRIBE	SUBSCRIBE
PUBLISH	PUBSUB	-	-
事务操作（5 个）			
MULTI	DISCARD	EXEC	UNWATCH
WATCH	-	-	-
运营维护类命令（8 个）			
SLOWLOG GET	SLOWLOG LEN	SLOWLOG RESET	MEMORY PURGE
MEMORY DOCTOR	MEMORY MALLOC-STATS	MEMORY USAGE	MEMORY STATS
Sentinel 哨兵服务支持（10 个）			
AUTH (active-code)	CLIENT	ROLE	SENTINEL
SUBSCRIBE	UNSUBSCRIBE	PSUBSCRIBE	PUNSUBSCRIBE
INFO	PING	-	-
Cluster 集群服务支持（3 个）			
CLUSTER INFO	CLUSTER SLOTS	CLUSTER NODES	-

4.2.2 测试例程

jedis 测试程序请参考附录 F JedisTest.java，基于 Jedis-3.0.0 测试通过。

4.3 TCP/IP方式客户端开发

4.3.1 命令请求

4.3.1.1 格式定义

命令请求格式定义如下：

sess_id command table_id key value <key1 key2 ... keyn>

其中：

- **sess_id**：流水号，必选项，该项只用于客户端校验服务器程序。服务程序收到流水号后不做其他处理，直接返回给客户端。如果客户端不需要校验，可以设置成固定值。
- **command**：命令，必选项。目前支持客户端调用的命令有 set（写数据）、get（按主键取 value 数据）、get<n>（按非唯一索引 keyn 取 value 数据，程序支持 3 个索引的联合查询，如果有多条会同时返回，各 value 间用空格分割）（例如：1 get1 ssslaab）、get<n>key<m>（按非唯一索引 keyn 取对应记录的非唯一索引 keym 的值，m=0 或缺省 m 时，取对应记录唯一索引 key 的值，如果有多条会同时返回，各返回值间用空格分隔）（例如：1 get1key 1 ssslaab）、del（删除指定的数据）。

- **table_id**: 需要操作的数据表的编号，与配置文件中的编号对应。
- **key**: 主键，必选项。**value** 数据的唯一索引。字符串，最大长度支持 128bytes。
- **value**: 值，可选项。字符串，最大长度为 512bytes。
- **key1**: **value** 的第一不唯一索引，可选项。
-
- **keyn**: **value** 的第 n 个不唯一索引，可选项。

4.3.2 命令说明

4.3.2.1 get 命令

根据输入条件取数据。输入条件可以是主键或索引，做联合索引查询时最多可以关联 3 个索引查询，返回的数据可以是 **value** 字段，也可以是主键或索引字段。

- 通过主键取 **value**: “get 主键值”。
- 通过索引 1 取所有匹配记录的 **value**: “get1 索引 1 值”。
- 通过索引 1、2 联合匹配取 **value**: “get1&2 索引 1 值 索引 2 值”。
- 通过索引 1 取所有匹配记录的索引 2: “get1key2 索引 1 值”。

举例，根据索引 1 和索引 2 查表 1 的 **value** 命令如下：

“1 get1&2 1 123 456”

4.3.2.2 set 命令

向系统内写数据。如果系统内无对应主键的数据则增加。如果已有对应的数据，要看 **updateModel** 配置，如果配置为 **insert** 则返回 28 号错误插入失败；否则更新现有记录。

4.3.2.3 upd 命令

向系统内更新数据，和 **set** 的区别是 **upd** 命令要求系统内已有对应的未过期的数据，否则会报 1 号错误更新失败。

upd 命令和 **set** 命令的区别是：1、**upd** 命令要求系统内必须有未过期的对应的数据；2、**upd** 命令与 **updateModel** 配置无关。

4.3.2.4 count 命令

根据输入的索引条件进行统计，并返回统计结果。输入条件只能是索引，可以最多 3 个索引做联合查询，命令格式如下：

- 根据索引 1 统计记录数：count1 索引 1 值。
- 根据索引 1、2 统计记录数：count1&2 索引 1 值 索引 2 值。

4.3.2.5 ttl (pttl) 命令

根据输入的 key 值查询指定记录的到期时间，ttl 返回距离到期时刻的秒数，pttl 返回举例到期时刻的毫秒数。返回-1 说明记录永不过期；返回-2 是记录没找到或已经过期。

4.3.2.6 del 命令

根据输入条件删除记录。输入条件可以是主键，也可以是索引。命令格式如下：

- 根据主键删除记录：del 主键。
- 根据索引 1 删除记录：del1 索引 1 值。
- 根据索引 1、2 删除记录：del1&2 索引 1 值 索引 2 值。

4.3.2.7 dump 命令

根据指定的索引条件输出 value 数据：

根据索引 1 查数据：dump1 索引 1 值。

4.3.2.8 incr 命令

根据输入的 key 值，对应的 value 进行增值操作。支持 value 的类型为 bytes、Int、Long 等，可成功执行的前提是 key 存在。可制定增加的数值，缺省是加 1。命令举例：

```
1 incr 1 123 2
```

4.3.2.9 decr 命令

根据输入的 key 值，对应的 value 进行减值操作。支持 value 的类型为 bytes、Int、Long 等，可成功执行的前提是 key 存在。可制定增加的数值，缺省是减 1。

4.3.2.10 list 命令组

实现类似 Redis 的 list 数据类型，程序增加 Variable 类型支持该功能。Variable 类型在 byte 数组的基础上增加 list 的支持。

当 list 中的总项数超过预定义数值（64 个）时，或者总长度超过 value 预设长度时，系统会创建一个 ArrayList 对象专门保存该项的数据。

list 命令组包含如下命令：

- **lpush**: 在原字符串左边叠加 (key 不存在时会新增)。如原来的 value 是"456", 执行 1 lpush 1 key 123 以后 value 变成"123\t456"。命令举例:

```
20 lpush 1 list1 two
```

```
21 lpush 1 list1 one
```

- **rpush**: 在原字符串右边叠加 (key 不存在时会新增)。命令举例:

```
22 rpush 1 list1 there
```

- **lpop**: 删除原字符串左起第一项, 并返回被去除项的内容。如果 value 中只剩最后 1 项, 返回该项, 并删除 key。下次同样的操作会返回“数据未找到”的错误。命令举例:

```
23 lpop 1 list1
```

- **rpop**: 删除原字符串右起第一项, 并返回被去除的项的内容。处理流程类似 lpop。命令举例:

```
24 rpop 1 list1
```

- **ltrim**: 只保留原字符串左起的指定项的文本, 后面的文本会被删除, 如果总项数小于输入值则 value 值无变化。例如, 只保留列表左边 5 项:

```
25 ltrim 1 list1 5
```

- **rtrim**: 只保留原字符串右起的指定项的文本。操作结果类似于 ltrim。

- **lfetch**: 取 list 的值。命令举例:

```
26 lfetch 1 list1
```

- **lcount**: 计算指定 key 对应的 value 的 list 个数。命令举例:

```
27 lcount 1 list1
```

4.3.2.11 set 命令组

实现类似 Redis 的 set 数据类型, 和 list 的区别: 1、集合中的各项不能重复 (去重); 2、无顺序。程序增加 Variable 类型支持该功能, Variable 类型在 byte 数组的基础上增加 set 的支持。

当 set 中的总项数超过预定义数值 (64 个) 时, 或者总长度超过 value 预设长度时, 系统会创建一个 hashset 对象专门保存该项的数据。

- **sadd**: 增加一项。举例:

```
31 sadd 1 s1 java
```

```
31 sadd 1 s1 go
```

```
31 sadd 1 s1 python
```

```
31 sadd 1 s1 c++
```

- **srem**: 删除指定项。举例: 32 srem s1 go
- **spop**: 随机删除 1 项, 并返回删除的内容。举例: 33 pop 1 s1
- **smembers**: 返回 set 中的所有项。举例: 34 smembers 1 s1
- **sismember**: 判断输入项是否在 set 中, 在则返回 1。
- **scard**: 返回当前 set 中的总项数。举例: 35 scard 1 s1

4.3.2.12 hash 命令组

实现类似 Redis 的 hash 数据类型, 程序增加 Variable 类型支持该功能。Variable 类型在 byte 数组的基础上增加 hash 的支持, 将字符 (\r) 作为字段的分隔符, 将 hash 内容转换为 “key ‘\r’ value” 的形式然后各项拼接到一起保存到 value 中。换言之, value 还是整段的 bytes 字符串, 当 hash 命令组操作时, 会按字符\r将整段字符串分解成各 key、value 项处理。

当 hash 中的总项数超过预定义数值 (64 个) 时, 或者总长度超过 value 预设长度时, 系统会创建一个 hashMap 对象专门保存该项的数据。

- **hset**: 设置 1 项。举例:
41 hset 1 h1 age 20
41 hset 1 h1 name jobs
- **hincrby**: 增加本集合内指定项的值 (整数)。举例: 42 hincrby 1 h1 age 10
- **hdel**: 删除本集合内的指定项。
- **hget**: 返回本集合中指定项的内容。
- **hgetall**: 返回本集合的全部内容, 返回格式为 key1、value1、key2、value2…。举例: 43 hgetall 1 h1
- **hlen**: 返回本集合中的总项数。
- **hkeys**: 返回本集合中的全部 key 值, 返回格式为: key1、key2、key3…。
- **hvals**: 返回本集合中全部 value 值, 返回格式同 hkeys 命令。
- **hexists**: 判断指定的 key 是否在本集合中。

4.3.2.13 zset 命令组

实现按给定分值排序的 set 数据结构。数据存储方式类似 hash 数据类型, 其中 key 为名称, value 保存分值。

排序集合 (zset) 支持的命令如下:

- **Zadd**: 向列表中添加。
- **Zcard**: 查列表总长度。

- **Zcount:** 计算指定分值区间内的成员数。
- **Zincrby:** 改变现有成员的分值。
- **Zinterstore:** 求 2 列表的交集并存到新列表中。
- **Zlexcount:** 根据 key 的范围计算区间内的成员数。
- **Zrange:** 返回指定下标区间内的成员。
- **Zrangebylex:** 返回 key 指定的区间范围内的成员数。
- **Zrangebyscore:** 返回指定分值范围内的成员数。
- **Zrank:** 返回指定成员的索引。
- **Zremrangebylex:** 删除 key 指定的区间范围内的成员。
- **Zremrangebyscore:** 删除分值 (score) 指定的区间范围内的成员。
- **Zremrangebyrank:** 删除下标指定的区间范围内的成员。
- **Zrevrange:** 逆序 (分值由高到低) 返回指定下标区间内的成员。
- **Zrevrangebylex:** 逆序 (分值由高到低) 返回 key 指定的区间范围内的成员数。
- **Zrevrank:** 逆序 (分值由高到低) 返回指定成员的索引。
- **Zscore:** 返回指定成员的分值。
- **Zunionstore:** 求 2 列表的并集并存到新列表中。

4.3.3 命令响应

请求相应格式定义如下:

response_id status < error_code error_msg> value <values...>

- **response_id:** 响应 ID, 对应命令请求中的 sess_id, 必填项。
- **status:** 命令执行结果状态: “ok” 为成功, “error” 为失败, 必填项。
- **error_code:** 当 status 为 “error” 时, 返回错误编码。编码定义见[错误代码表](#)。
- **value:** 查询命令成功时返回 value 数据值。可选项, 查询命令的返回内容。
- **<values...>:** 查询多项结果时返回其他的 value 值, 个项用空格分隔。可选项。

4.3.4 命令用例

命令用例如下:

```
# upd 命令更新需要先有数据

1 upd 1 aaa bbb 111 222 333

1 error 29 error in upd

# set 写数据

1 set 1 aaa ccc 112 223 334

1 ok

# get 读数据

1 get 1 aaa

1 ok ccc

# upd 更新数据

1 upd 1 aaa bbb 111 222 333

1 ok

1 get 1 aaa

1 ok bbb

# 写数据

1 set 1 bbb ccc 111 333 333

1 ok

1 set 1 ccc ddd 111 444 555

1 ok

1 set 1 eee fff 222 333 333

1 ok

# 根据索引 1 查询

1 get1 1 111

1 ok bbb ccc ddd

# 根据索引 3 查询

1 get3 1 333

1 ok bbb ccc fff

# 根据索引 1、索引 3 联合查询
```

```

1 get1&3 1 111 333

1 ok bbb ccc

# dump 根据索引 1 取 value 数据

1 dump1 1 111

bbb

ccc

ddd

dump ok

# dump 根据索引 3 取 value 数据

1 dump3 1 333

bbb

ccc

fff

dump ok

```

4.3.5 错误代码表

错误代码	错误原因
0	操作成功
-1	请求格式非法
-2	请求中的 command 参数为空
-3	请求中的 key 参数为空
-4	不能解析请求的命令
-5	table_id 参数非法
-6	set、upd 命令时 value 为空
-7	set、upd 命令时索引为空
-9	异常错
-10	系统未启动完成，不接收同步以外的命令
1	未找到需要查询的数据
2	写/更新数据（set）错误
21	key 值超长
22	value 值超长
23	某个 index 超长，具体需要看 server 的日志
26	写数据时加锁失败
27	服务器数据更新方式配成 insert 方式（Server.Table<n>.updateModel = insert），且输

错误代码	错误原因
	入数据的 key 在系统中已存在
28	服务器自动覆盖方式配成 false，不自动覆盖较旧的数据（Server.AutoOverlay = false），且内存表已满
3	删除数据（del）错误
4	匹配到的数据过多
41	同步数据比内存表中的数据旧，不更新
42	同步的数据已经被删除,不允许插入

4.3.6 命令操作举例

4.3.6.1 设置 key= “str1”，value= “test”

命令：

```
10 set 1 str1 test
```

其中：

- **10**：是 session-id，帮助客户端校验请求和响应的对应关系（通常是客户端自己创建一个流水号）；set 是设置字符串的命令；
- **1**：是第一张表（对应配置文件的 table1）；
- **str1**：是 key 的值；
- **test**：是 value 的值，各项用空格分隔，整条命令用\n 结束。

响应

```
10 ok
```

其中：**10** 是客户端发起的流水号；**ok** 表示设置成功。

4.3.6.2 查询 key= “str1” 的 value 值

命令：

```
11 get 1 str1
```

响应：

```
11 ok test
```

4.3.7 特殊命令

特殊命令为简单命令格式，无序列号，也没有其它参数。

4.3.7.1 heartbeat

心跳（heartbeat）命令可用于客户端检测服务器是否可用，如果服务器已经启动完成并可提供服务，收到该命令时将返回“hb”+服务器启动完成时的时间；否则服务器会断开当前连接。用例如下：

```
test[mysql:/data1/mysql]=> telnet 0 16200

Trying 0.0.0.0...

Connected to 0.

Escape character is '^]'.

heartbeat

hb 1400559146481
```

4.3.7.2 check

check 命令用于内部的数据同步线程检测服务器是否能正常监听端口，服务器收到该命令后返回“ck”+服务器开始启动的时间+服务器唯一标志（唯一标志格式为“16 进制 ID 号-端口号”+redis 协议端口号，同步线程用该标志判断当前连接的对端是否是自己，如果是自己则该连接不需要同步数据）。该命令只有同步包中的 Connector 类使用。用例如下：

```
test[mysql:/data1/mysql/memdb/bin]=> telnet 0 16200

Trying 0.0.0.0...

Connected to 0.

Escape character is '^]'.

check

ck 1467963869641 79a80003ba2c8aa6-16200 6379
```

4.3.7.3 Idle

idle 命令只用于保持连接。如果客户端长时间没有传输数据，可以用该命令定期检测连接状态，并保持连接。服务器收到该命令后返回“ok”。

4.3.7.4 dump

dump 命令用于持久化当前内存中的数据。程序会在主目录的 var 子目录下创建文件名为 **dump-{table-id}.dat** 的文件组，对应内存中的表。文件采用 BerkeleyDB 格式存储，该文件以 key-value 对的形式保存数据。

4.3.7.5 exit

退出（exit）命令用于终止当前连接，服务器收到该命令后会立即关闭当前连接。用例如下：

```
test[mysql:/data1/mysql]=> telnet 0 16200

Trying 0.0.0.0...

Connected to 0.

Escape character is '^]'.

exit

Connection to 0 closed by foreign host.

test[mysql:/data1/mysql]=>
```

4.3.8 客户端API演示程序

本程序样例实现了 Socket 连接、集群多节点异常切换等功能。实现了最常用的 set 和 get 接口。

```
import java.io.*;
import java.net.*;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Clinet 演示
 */
public class ClientDemo {
    private final InetSocketAddress[] Addresses;

    private final String Password;

    private int CurrentAddr = 0;

    private Socket CurrentSocket = null;
    private Writer CurrentWriter = null;
    private BufferedReader CurrentReader = null;

    private AtomicLong SessionId = new AtomicLong(0);

    /**
     * @param passwd 如果 Server 端设置了需要密码认证，此处为密码，否则要必须为 null
     */
}
```

```

    * @param addr 服务器的地址列表
    */
    public ClientDemo(String passwd, InetAddress... addr) {
        if (addr == null || addr.length <= 0) {
            throw new IllegalArgumentException("Address is null");
        }

        Password = passwd;
        Addresses = addr;

        exchange();
    }

    private void exchange() {
        if (CurrentSocket != null) {
            try {
                CurrentSocket.close();
            } catch (Throwable e) {
                //e.printStackTrace();
            }
        }
        for (int i = 0; i < Addresses.length; i++) {
            try {
                CurrentSocket = new Socket();
                CurrentSocket.setTcpNoDelay(true);
                CurrentSocket.setSoTimeout(1000); // 1 second
                CurrentSocket.connect(Addresses[CurrentAddr], 1000);
                CurrentWriter = new OutputStreamWriter(CurrentSocket.getOutputStream());
                if (Password != null) {
                    CurrentWriter.write(Password);
                    CurrentWriter.write('\n');
                    CurrentWriter.flush();
                }
                CurrentReader = new BufferedReader(new InputStreamReader(CurrentSocket.getInp
utStream()));
                break;
            } catch (Exception e) {
                //e.printStackTrace();
            }
            CurrentAddr = (CurrentAddr + 1) % Addresses.length;
        }
    }
}

```

```

/**
 * set 命令
 *
 * @param key
 * @param value
 * @return
 */
public synchronized boolean set(String key, String value) {
    String sess_id = Long.toString(SessionId.incrementAndGet());
    String resp = null;
    try {
        CurrentWriter.write(sess_id + " set 1 " + key + " " + value + "\n");
        CurrentWriter.flush();
        resp = CurrentReader.readLine();
    } catch (Exception e1) {
        exchange();
        try {
            CurrentWriter.write(sess_id + " set 1 " + key + " " + value + "\n");
            CurrentWriter.flush();
            resp = CurrentReader.readLine();
        } catch (Exception e) {
        }
    }

    return resp != null && resp.startsWith(sess_id + " ok");
}

/**
 * get 命令
 *
 * @param key
 * @return
 */
public synchronized String get(String key) {
    String sess_id = Long.toString(SessionId.incrementAndGet());
    String resp = null;
    try {
        CurrentWriter.write(sess_id + " get 1 " + key + "\n");
        CurrentWriter.flush();
        resp = CurrentReader.readLine();
    } catch (Exception e1) {

```

```

        exchange();
        try {
            CurrentWriter.write(sess_id + " get 1 " + key + "\n");
            CurrentWriter.flush();
            resp = CurrentReader.readLine();
        } catch (Exception e) {
        }
    }
    String head = sess_id + " ok ";
    if (resp != null && resp.startsWith(head)) {
        return resp.substring(head.length());
    }

    return null;
}

public static void main(String[] argv) {
    InetAddress addr1 = new InetAddress("110.50.1.55", 16200);
    InetAddress addr2 = new InetAddress("110.50.1.56", 16200);
    InetAddress addr3 = new InetAddress("110.50.1.57", 16200);

    ClientDemo client = new ClientDemo(null, addr1, addr2, addr3);

    boolean ret = client.set("key1", "value1");

    System.out.println(client.get("key1"));
}
}

```

附录A SetSplit2MultiModule.java

插件源码如下：

```
package com.memdb.modules;

import com.memdb.util.Interaction;
import com.memdb.util.PluginModule;

import java.nio.charset.StandardCharsets;
import java.util.*;

/**
 * 该插件拦截 hash 类操作命令，将一个 key 下面的项分配到多个 key 下面存储，解决单 key 对应的
 * 数据量过大问题
 */
public class SetSplit2MultiModule implements PluginModule {
    // 缺省分块数
    private final static int DEFAULT_BLOCKS = 16;
    // 缺省的 smembers 命令返回的最多元素数，超过此值返回失败
    private final static int DEFAULT_MAXITMES = 1024 * 100;
    // 缺省的 block 值在 sscan 命令返回结果中的 cursor 中保存的起始位置（bit 位），packed 模式时
    // 小于 32
    private final static int DEFAULT_BLOCKBITPOSITION = 32;

    private int BLOCKS = DEFAULT_BLOCKS;
    private String separator = "^o^";
    private int maxItems = DEFAULT_MAXITMES;

    // 以下两项只用户 sscan 命令时返回 cursor 中的 block 信息和 block 内的 cursor 信息的组合
    // 返回的 cursor 中 block 信息的起始位置（比特数）
    private int blockBitPosition = DEFAULT_BLOCKBITPOSITION;
    // 返回 cursor 中 block 内的 cursor 值的掩码
    private long dataMask = (1l << blockBitPosition) - 1;

    @Override
    public void init(Properties properties) {
        String value;
        value = properties.getProperty("suffix");
        if (value != null && value.length() > 0) {
            this.separator = value;
        }
    }
}
```

```

        value = properties.getProperty("blocks");
        if (value != null) {
            try {
                BLOCKS = Integer.parseInt(value);
                if (BLOCKS <= 1) {
                    BLOCKS = 2;
                }
            } catch (Throwable t) {
                BLOCKS = DEFAULT_BLOCKS;
            }
        }
        if ("true".equalsIgnoreCase(properties.getProperty("packedblocknumber"))) {
            // 此配置下理论上最多能存储的项是 2 的 32 次方减一。
            // 但采用此配置时 sscan 返回 cursor 值的范围的兼容性更好
            // 该配置只影响 sscan 命令
            int bit = 0;
            while ((1 << bit) < BLOCKS) {
                bit++;
            }
            blockBitPosition = 32 - bit;
            dataMask = (11 << blockBitPosition) - 1;
        }

        value = properties.getProperty("maxItems");
        if (value != null) {
            try {
                maxItems = Integer.parseInt(value);
            } catch (Throwable t) {
                maxItems = DEFAULT_MAXITEMS;
            }
        }
    }

    @Override
    public String[] getCommands() {
        return new String[]{"sadd", "scard", "sdiff", "sdiffstore", "sinter", "sinterstore", "sunion",
            "sunionstore",
            "sismember", "smembers", "smove", "spop", "srandmember", "srem", "sscan", "del",
            "type"};
    }

```



```

@Override
public Object process(List in, Interaction interaction, int db) {
    Object ret = null;
    String cmd = (String) in.get(0);
    interaction.log(Interaction.LOGLEVEL.DEBUG, "receive '" + cmd + "'");
    if ("type".equals(cmd)) {
        return getType((byte[]) in.get(1), interaction, db);
    } else if ("del".equals(cmd)) {
        int count = 0;
        for (int i = 1; i < in.size(); ++i) {
            byte[] key = (byte[]) in.get(i);
            if ("set".equalsIgnoreCase(getType(key, interaction, db))) {
                del(key, interaction, db);
                in.remove(i);
                count++;
            }
        }
    }
    if (in.size() > 1) {
        Long l = (Long) interaction.call(in, db);
        count += l.intValue();
    }
    return new Long(count);
} else if ("sismember".equals(cmd)) {
    // 双目命令
    byte[] key = (byte[]) in.get(1);
    byte[] field = (byte[]) in.get(2);
    ArrayList req = new ArrayList();
    req.add(cmd);
    req.add(getNewKey(key, field));
    req.add(field);
    ret = interaction.call(req, db);
} else if ("smove".equals(cmd)) {
    // 三目命令
    byte[] old_key = (byte[]) in.get(1);
    byte[] new_key = (byte[]) in.get(2);
    byte[] field = (byte[]) in.get(3);
    int block = getHash(field, BLOCKS);
    ArrayList req = new ArrayList();
    req.add("srem");
    req.add(getNewKey(old_key, block));
    req.add(field);
    ret = interaction.call(req, db);
}

```

```

        if (ret instanceof Long && ((Long) ret).longValue() == 1) {
            req.clear();
            req.add("sadd");
            req.add(getNewKey(new_key, block));
            req.add(field);
            interaction.call(req, db);
            ret = new Long(1);
        }
    } else if ("smembers".equals(cmd)) {
        // 单目命令
        byte[] key = (byte[]) in.get(1);
        if (maxItems > 0) {
            int len = approximateLen(interaction, key, db);
            if (len > maxItems) {
                return new IllegalArgumentException("Err too many fields were found");
            }
        }

        ArrayList[] reqs = new ArrayList[BLOCKS];
        for (int i = 0; i < BLOCKS; ++i) {
            // 创建 key 的数组
            reqs[i] = new ArrayList();
            reqs[i].add(cmd);
            reqs[i].add(getNewKey((byte[]) in.get(1), i));
        }
        for (int i = 0; i < BLOCKS; ++i) {
            Object o = interaction.call(reqs[i], db);
            if (o instanceof List) {
                if (ret == null) {
                    ret = new ArrayList<>();
                }
                List list = (List) o;
                if (list.size() > 0) {
                    ((List) ret).addAll(list);
                }
            } else if (o instanceof Map) {
                // for RSP3
                if (ret == null) {
                    ret = new HashMap<>();
                }
                Map map = (Map) o;
                if (map.size() > 0) {

```

```

                ((Map) ret).putAll(map);
            }
        }
    }
} else if ("spop".equals(cmd)) {
    // 单目命令
    byte[] key = (byte[]) in.get(1);
    ArrayList req = new ArrayList();
    req.add(cmd);
    req.add("");
    for (int i = 0; i < BLOCKS; ++i) {
        req.set(1, getNewKey(key, i));
        ret = interaction.call(req, db);
        if (ret != null) {
            break;
        }
    }
} else if ("scard".equals(cmd)) {
    // 单目命令
    byte[] key = (byte[]) in.get(1);
    long len = 0;
    ArrayList req = new ArrayList();
    req.add(cmd);
    req.add("");
    for (int i = 0; i < BLOCKS; ++i) {
        req.set(1, getNewKey(key, i));
        Object o = interaction.call(req, db);
        if (o instanceof Long) {
            len += (Long) o;
        } else if (o instanceof Exception) {
            return o;
        }
    }
    ret = new Long(len);
} else if ("srem".equals(cmd)) {
    // 多目命令
    byte[] key = (byte[]) in.get(1);
    ArrayList[] reqs = new ArrayList[BLOCKS];
    for (int i = 0; i < BLOCKS; ++i) {
        reqs[i] = new ArrayList();
        reqs[i].add(cmd);
        reqs[i].add(getNewKey(key, i));
    }
}

```

```

    }
    for (int i = 2; i < in.size(); ++i) {
        reqs[getHash((byte[]) in.get(i), BLOCKS)].add(in.get(i));
    }
    long deled = 0;
    for (int i = 0; i < BLOCKS; ++i) {
        if (reqs[i].size() > 2) {
            try {
                deled += (Long) interaction.call(reqs[i], db);
            } catch (Throwable t) {
            }
        }
    }
    ret = new Long(deled);
} else if ("sadd".equals(cmd)) {
    // 多目命令
    long len = 0;
    byte[] key = (byte[]) in.get(1);
    ArrayList[] reqs = new ArrayList[BLOCKS];
    for (int i = 0; i < BLOCKS; ++i) {
        // 创建 key 的数组
        reqs[i] = new ArrayList();
        reqs[i].add(cmd);
        reqs[i].add(getNewKey(key, i));
    }
    for (int i = 2; i < in.size(); ++i) {
        int block = getHash((byte[]) in.get(i), BLOCKS);
        reqs[block].add(in.get(i));
    }
    for (int i = 0; i < BLOCKS; ++i) {
        if (reqs[i].size() > 2) {
            Object o = interaction.call(reqs[i], db);
            if (o instanceof Long) {
                len += ((Long) o).longValue();
            } else {
                return o;
            }
        }
    }
    ret = new Long(len);
} else if ("sscan".equals(cmd)) {
    // 多目可变参数命令

```

```

int count = 10;
byte[] match = null;
try {
    for (int i = 3; i < in.size(); ++i) {
        String arg = new String((byte[]) in.get(i));
        if ("count".equalsIgnoreCase(arg)) {
            try {
                count = Integer.parseInt(new String((byte[]) in.get(i + 1)));
            } catch (Exception e) {
            }
        } else if ("match".equalsIgnoreCase(arg)) {
            try {
                match = (byte[]) in.get(i + 1);
            } catch (Exception e) {
            }
        }
    }
} catch (Throwable t) {
}

long cursor = Long.parseLong(new String((byte[]) in.get(2)));
int block = (int) (cursor >>> blockBitPosition);
cursor = cursor & dataMask;
ArrayList req = new ArrayList();
int leaved = count;
int i;
for (i = block; i < BLOCKS; ++i) {
    ArrayList req = new ArrayList();
    req.add(cmd);
    req.add(getNewKey((byte[]) in.get(1), i));
    req.add(Long.toString(cursor).getBytes(StandardCharsets.UTF_8));
    if (match != null) {
        req.add("match".getBytes(StandardCharsets.UTF_8));
        req.add(match);
    }
    if (count > 0) {
        req.add("count".getBytes(StandardCharsets.UTF_8));
        req.add(Integer.toString(leaved).getBytes(StandardCharsets.UTF_8));
    }
    Object o = interaction.call(req, db);
    if (o instanceof List) {
        List list = (List) o;
        cursor = Long.parseLong(new String((byte[]) list.get(0)));
    }
}

```

```

        resq.addAll((List) list.get(1));

        // 当前 block 的项数太多了
        if (cursor > dataMask) {
            return new IllegalStateException("Err too many items(" + cursor + ") in "
                + new String(getNewKey((byte[]) in.get(1), i))
                + ", the max is " + dataMask);
        }

        if (cursor > 0 || count > 0 && resq.size() >= count) {
            // 本次已经结束了
            break;
        }
    } else if (o instanceof Exception) {
        return o;
    }
}

if (cursor == 0) {
    // 当前 block 已经全部取完了，代表当前 block 的值需要+1
    i++;
}

if (i < BLOCKS || cursor > 0) {
    long l = i == BLOCKS ? 0 : i;
    cursor = (l << blockBitPosition) + cursor;
}

List response = new ArrayList();
response.add(Long.toString(cursor));
response.add(resq);
ret = response;
} else {
    ret = new IllegalAccessException("Err unsupport command '" + cmd + "' in plugin
SetSplit2MultiModule");
}

interaction.log(Interaction.LOGLEVEL.DEBUG, "request: " + cmd + ", response: " + ret);
return ret;
}

/**
 * 大致计算当前 map 中有多少条记录
 *
 * @param interaction
 * @param key

```

```

    * @param db
    * @return
    */
    private int approximateLen(Interaction interaction, byte[] key, int db) {
        long len = 0;
        ArrayList req = new ArrayList();
        req.add("scard");
        req.add("");
        int loop = 0;
        for (int i = 0; i < BLOCKS; i += 7) {
            req.set(1, getNewKey(key, i));
            len += (Long) interaction.call(req, db);
            loop++;
        }
        if (loop > 1) {
            len = (long) (len * 1.0d / loop * BLOCKS);
        }
        return (int) len;
    }

    private String getType(byte[] key, Interaction interaction, int db) {
        ArrayList list = new ArrayList<>();
        list.add("type");
        list.add("");
        for (int i = 0; i < BLOCKS; ++i) {
            list.set(1, getNewKey(key, i));
            Object o = interaction.call(list, db);
            if (o instanceof String) {
                String resp = (String) o;
                if (!"none".equalsIgnoreCase(resp)) {
                    return resp;
                }
            }
        }
        list.set(1, key);
        return (String) interaction.call(list, db);
    }

    private void del(byte[] key, Interaction interaction, int db) {
        ArrayList list = new ArrayList<>();
        list.add("del");
        for (int i = 0; i < BLOCKS; ++i) {

```

```

        list.add(getNewKey(key, i));
    }
    interaction.call(list, db);
}

private byte[] getNewKey(byte[] key, byte[] field) {
    return getNewKey(key, getHash(field, BLOCKS));
}

private byte[] getNewKey(byte[] key, int block) {
    StringBuilder buf = new StringBuilder();
    buf.append(new String(key, StandardCharsets.UTF_8)).append(separator).append(block);
    return buf.toString().getBytes(StandardCharsets.UTF_8);
}
}

```


附录B TestClient.java

样例程序如下：

```
package com.rds.client.test;

import com.rds.client.javaclient.Client;
import com.rds.client.javaclient.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient {

    public static void main(String[] argv) throws IOException {

        //      InetSocketAddress addr1 = new InetSocketAddress("localhost", 6300);
        //      InetSocketAddress addr2 = new InetSocketAddress("localhost", 6301);
        //      InetSocketAddress addr3 = new InetSocketAddress("localhost", 6302);
        //
        //      //ConnectionPool pool = new ConnectionPool("WebSession", "acioweor_483kja03np4h8238G",
        //      addr1, addr2, addr3);
        //      //Connection conn = pool.getConnection();
        //
        //      CenterClient client = new CenterClient("WebSession", "acioweor_483kja03np4h8238G", addr1,
        //      addr2, addr3);

        Client client = new Client(false, false, null
            , new InetSocketAddress("192.168.0.80", 6200)
            , new InetSocketAddress("192.168.0.81", 6200));

        Connection conn = client.getConnection();

        //      try {
        //          Thread.sleep(1000000);
        //      } catch (InterruptedException e) {
        //          e.printStackTrace();
        //      }
        boolean ret;
        ret = conn.set("key1", "value1");
        ret = conn.set("key2", "value2");
        ret = conn.set("key3", "value3");
    }
}
```

```

ret = conn.set("key4", "value4");
ret = conn.exist("key1");
ret = conn.exist("key5");

System.out.println(conn.get("key1"));

System.out.println("changed key1: " + conn.getchg("key1") + "");
System.out.println("changed key1: " + conn.getchg("key1") + "");
System.out.println("changed key0: " + conn.getchg("key0") + "");

conn.hset("h1", "name", "july");
conn.hset("h1", "age", "0");

String value = "User(id=1, userName=eysi tesys, password=$2n$s&*\\g/aeweoie...sw, real=...";
String value2 = "User(id=1, userName=azefz, pass\\nword=$2e$swq*\\g/aewfs      asfj  w
as;ifjw a;wiejfooiej09309384 af      eoie.ewrsw, realTime=2011";
conn.hset("cache-auth-user", "eysi tesys", value);
conn.hset("cache-auth-user", "azefz", value2);
System.out.println("\\nhash:");
System.out.println("eysi tesys: " + conn.hget("cache-auth-user", "eysi tesys"));
System.out.println("azefz: " + conn.hget("cache-auth-user", "azefz"));
for (String v : conn.hgetAll("cache-auth-user")) {
    System.out.println(v);
}
System.out.println(conn.hdel("h1", "age"));

System.out.println("\\nstring:");
conn.set("cache-auth-user-ezsitesys", value);
System.out.println(conn.get("cache-auth-user-ezsitesys"));

System.out.println("\\nset:");
conn.sadd("cache-auth-user-ezsitesys1", value);
for (String v : conn.smembers("cache-auth-user-ezsitesys1")) {
    System.out.println(v);
}

conn.close();

for (int i = 0; i < 10000000; i++) {

    //conn = pool.getConnection();

```

```

conn.setnx("str", "string");
conn.exist("str");
conn.append("str", "-strings");
System.out.println(conn.substr("str", 0, 4));
conn.del("str");

for (String s : conn.mget("key4", "key1", "key2", "key3")) {
    System.out.print(s + " ");
}
System.out.print('\n');

conn.sadd("s1", "www");
conn.sadd("s1", "http");
conn.sadd("s1", "https");
for (String s : conn.smembers("s1")) {
    System.out.print(s + " ");
}
System.out.print('\n');

conn.hsetnx("h1", "name", "july");
conn.hincr("h1", "age", 1);
for (String s : conn.hgetAll("h1")) {
    System.out.print(s + " ");
}
System.out.print('\n');

conn.close();

try {
    Thread.sleep(10);
} catch (InterruptedException e) {
    e.printStackTrace();
}
}
}
}

```

附录C TestClientPooled.java

样例程序如下：

```
/**
 * 采用连接池方式连接数据服务节点
 */
package com.rds.client.test;

import com.rds.client.javaclient.PooledClient;
import com.rds.client.javaclient.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClientPooled {
    public static void main(String[] arg) throws IOException {
        // ConnectionPool(boolean useSsl, boolean needPwd, String password, InetSocketAddress... addrs)
        PooledClient pool = new PooledClient(false, false, null
            , new InetSocketAddress("192.168.0.60", 6200)
            , new InetSocketAddress("192.168.0.60", 6201));

        Connection connection = pool.getConnection();

        connection.set("str-key1", "str-value1");
        connection.set("str-key2", "str-value2");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();

        connection = pool.getConnection();

        System.out.println("value = " + connection.get("str-key2"));

        connection.close();

        pool.close();
    }
}
```

附录D TestClientCenter.java

样例程序如下：

```
/**
 * 通过给出的中心节点获得服务节点的连接信息，包括服务节点的地址、端口、通道加密方式、密码等，创建到服务节点的连接池
 * （企业版支持此功能）
 */
package com.rds.client.test;

import com.rds.client.javaclient.CenterClient;
import com.rds.client.javaclient.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClientCenter {
    public static void main(String[] arg) throws IOException {
        CenterClient client = new CenterClient("WebSession", "acioweor_483kja03np4h8238G"
            , new InetSocketAddress("192.168.0.60", 6300)
            , new InetSocketAddress("192.168.0.60", 6301));

        Connection connection = client.getConnection();

        connection.set("str-key1", "str-value1");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}
```

附录E 程序样例

TestClient1.java:

```
/**
 * 直接连接数据服务节点
 */
package com.agk.memdb;

import com.agk.memdb.javaclient.Client;
import com.agk.memdb.javaclient.conn.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient1 {
    public static void main(String[] arg) throws IOException {
        Client client = new Client(false, false, null
            , new InetSocketAddress("192.168.0.60", 6200)
            , new InetSocketAddress("192.168.0.60", 6201));

        Connection connection = client.getConnection();

        connection.set("str-key1", "str-value1");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}
```

TestClient2.java:

```
/**
 * 采用连接池方式连接数据服务节点
 */
package com.agk.memdb;

import com.agk.memdb.javaclient.ConnectionPool;
import com.agk.memdb.javaclient.conn.Connection;
```

```
import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient2 {
    public static void main(String[] arg) throws IOException {
        ConnectionPool pool = new ConnectionPool(false, false, null
            , new InetSocketAddress("192.168.0.60", 6200)
            , new InetSocketAddress("192.168.0.60", 6201));

        Connection connection = pool.getConnection();

        connection.set("str-key1", "str-value1");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}
```

TestClient3.java:

```
/**
 * 通过给出的中心节点获得对应服务节点的连接信息，包括服务节点的地址、端口、通道加密方式、
 * 密码等，得到服务的访问连接
 */
package com.agk.memdb;

import com.agk.memdb.javaclient.Client;
import com.agk.memdb.javaclient.conn.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient3 {
    public static void main(String[] arg) throws IOException {
        Client client = new Client("WebSession","acioweor_483kja03np4h8238G"
            , new InetSocketAddress("192.168.0.60", 6300)
            , new InetSocketAddress("192.168.0.60", 6301));

        Connection connection = client.getConnection();

        connection.set("str-key1", "str-value1");
    }
}
```

```

        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}

```

TestClient4.java:

```

/**
 * 通过给出的中心节点获得服务节点的连接信息，包括服务节点的地址、端口、通道加密方式、密码等，创建到服务节点的连接池
 */
package com.agk.memdb;

import com.agk.memdb.javaclient.ConnectionPool;
import com.agk.memdb.javaclient.conn.Connection;

import java.io.IOException;
import java.net.InetSocketAddress;

public class TestClient4 {
    public static void main(String[] arg) throws IOException {
        ConnectionPool pool = new ConnectionPool("WebSession", "acioweor_483kja03np4h8238G"
            , new InetSocketAddress("192.168.0.60", 6300)
            , new InetSocketAddress("192.168.0.60", 6301));

        Connection connection = pool.getConnection();

        connection.set("str-key1", "str-value1");
        System.out.println("value = " + connection.get("str-key1"));

        connection.close();
    }
}

```


附录F JedisTest.java

jedis 测试程序如下：

```
package com.agk.memdb;

import org.apache.commons.pool2.impl.GenericObjectPoolConfig;
import redis.clients.jedis.Jedis;
import redis.clients.jedis.JedisSentinelPool;

import java.util.HashMap;
import java.util.HashSet;

public class JedisTest {
    public static void main(String[] argv) {
        JedisSentinelPool pool = null;
        Jedis jedis = null;
        try {
            pool = new JedisSentinelPool("WebSession", new HashSet<String>() {{
                this.add("192.168.0.60:26379");
            }}, (String) null, "aciowear_483kja03np4h8238G");

            jedis = pool.getResource();
        } catch (Exception e) {
            System.out.println(e.toString());
            //jedis = new Jedis("192.168.0.60", 6379);
        }

        System.out.println("Sentinal...");

        /**
         * 以下为通用功能测试，共 22 个
         */
        System.out.println("Common...");
        System.out.println(jedis.set("aaa", "12345"));
        System.out.println(jedis.setnx("ddd", "kkkkk"));
        System.out.println(jedis.setex("aaa", 7, "123"));
        System.out.println(jedis.exists("aaa"));
        System.out.println(jedis.get("aaa"));
        System.out.println(jedis.substr("aaa", 1, 3));
        System.out.println(jedis.getSet("eee", "I"));
        System.out.println(jedis.append("eecaa", "fff"));
    }
}
```

```

System.out.println(jedis.pexpire("aaa", 5000));
System.out.println(jedis.expire("aaa", 14));
System.out.println(jedis.expireAt("eee", 12345678900l));
System.out.println(jedis.pexpireAt("eee", 1234567890000l));
System.out.println(jedis.persist("aaa"));
System.out.println(jedis.incr("aaa"));
System.out.println(jedis.incrBy("aaa", 5));
System.out.println(jedis.decr("aaa"));
System.out.println(jedis.decrBy("aaa", 5));
System.out.println(jedis.incrByFloat("aaa", -2.5));
System.out.println(jedis.ttl("aaa"));
System.out.println(jedis.psetex("aaa", 5432, "100"));
System.out.println(jedis.pttl("aaa"));
System.out.println(jedis.del("l1", "h1", "s1"));

/**
 * 以下为list 类型数据操作测试（暂不支持lrem），11 个
 */
System.out.println("\nList...");
System.out.println(jedis.lpush("l1", "aaa", "bb"));
System.out.println(jedis.rpush("l1", "ccc", "ddd"));
System.out.println(jedis.lpop("l1"));
System.out.println(jedis.rpop("l1"));
System.out.println(jedis.lindex("l1", 1));
System.out.println(jedis.lpushx("l1", "123", "345"));
System.out.println(jedis.rpushx("l1", "eee", "fff"));
System.out.println(jedis.lset("l1", 1, "abc"));
System.out.println(jedis.lrange("l1", 1, 3));
System.out.println(jedis.llen("l1"));
System.out.println(jedis.ltrim("l1", 0, 6));

/**
 * 以下为set 类型数据操作测试（暂不支持sscan），12 个
 */
jedis.del("s1", "s2", "s3");
jedis.sadd("s1", "aaa", "bbb", "ccc", "ddd");
jedis.sadd("s2", "ccc", "ddd", "eee", "fff");
System.out.println("\nSet...");
System.out.println(jedis.sadd("s3", "aa", "bbbbbb", "ccc", "ddddddd", "eedde"));
System.out.println(jedis.smembers("s3"));
System.out.println(jedis.sismember("s3", "aaa"));
System.out.println(jedis.srem("s3", "aa"));

```

```

System.out.println(jedis.spop("s3"));
System.out.println(jedis.spop("s3", 2));
System.out.println(jedis.scard("s3"));
System.out.println(jedis.sdiff("s1", "s2"));
System.out.println(jedis.sdiffstore("s3", "s44", "s1"));
System.out.println(jedis.sinter("s1", "s2"));
System.out.println(jedis.sinterstore("s3", "s1", "s2"));
System.out.println(jedis.sunion("s1", "s2"));
System.out.println(jedis.sunionstore("s3", "s1", "s2"));

/**
 * 以下为 hash 类型数据操作测试（暂不支持 hscan 命令），12 个
 */
System.out.println("\nHash...");
HashMap<String, String> h1 = new HashMap<String, String>() {{
    this.put("name", "jone");
    this.put("age", "10");
}};
System.out.println(jedis.hmset("h1", h1));
System.out.println(jedis.hset("h1", "name", "tomy"));
System.out.println(jedis.hget("h1", "name"));
System.out.println(jedis.hgetAll("h1"));
System.out.println(jedis.hexists("h1", "name"));
System.out.println(jedis.hkeys("h1"));
System.out.println(jedis.hvals("h1"));
System.out.println(jedis.hincrBy("h1", "age", 1));
System.out.println(jedis.hincrByFloat("h1", "age", 1));
System.out.println(jedis.hsetnx("h1", "job", "no"));
System.out.println(jedis.hlen("h1"));
System.out.println(jedis.hstrlen("h1", "job"));

jedis.close();
if (pool != null) {
    pool.close();
}
}
}

```

附录G 术语说明

名称	说明
TongRDS (简称 RDS)	东方通分布式内存数据缓存中间件
中心节点	提供服务节点注册、运行模式管理功能的管理节点
服务节点	实际提供数据缓存服务的功能节点

